Noise-Dependent Planning for Discrete Diffusion LLMs

Design Document: From Block Diffusion to Lightweight Coarse-to-Fine Generation

Contents

1 Core Thesis **2**

2 Background: What Block Diffusion Actually Does **2**

3 Mechanism C: Noise-Dependent Structured Masking (P0—Primary) **3**
- 3.1 The Problem with Uniform Random Masking . . . 3
- 3.2 Core Design . . . 3
- 3.3 Formal Specification . . . 3
- 3.4 Implementation Steps . . . 4
- 3.5 Key Hyperparameters . . . 5
- 3.6 Why This Should Work (Connection to Block Diffusion) . . . 5

4 Mechanism A: Noise-Dependent Loss Granularity (P1—Secondary) **5**
- 4.1 Motivation . . . 5
- 4.2 Core Design . . . 6
- 4.3 What Is the Span-Level Loss? . . . 6
- 4.4 Recommended Implementation: Start with A1, Prepare A2 . . . 6
- 4.5 Why A Complements C . . . 7

5 Mechanism B: Noise-Dependent Attention Routing (P2—Exploratory) **8**
- 5.1 Motivation . . . 8
- 5.2 Core Design . . . 8
- 5.3 Specification . . . 8
- 5.4 Implementation Concerns . . . 8
- 5.5 When to Pursue B . . . 9

6 Experimental Plan **9**
- 6.1 Ablation Structure . . . 9
- 6.2 Evaluation: Tasks Where Planning Matters . . . 9
- 6.3 Diagnostic Metrics . . . 9

7 Implementation Roadmap **10**

8 Risk Assessment **10**

1 Core Thesis

Standard discrete diffusion language models (dLLMs) such as MDLM and SEDD use uniform random masking at all noise levels. This means the model never receives an explicit training signal to separate *global planning* (what to say) from *local infilling* (how to say it). Block Diffusion demonstrated that explicitly decoupling these two scales—block-level coarse generation followed by token-level refinement—yields significant gains. However, Block Diffusion achieves this through architectural hierarchy (a two-tier diffusion process), which introduces structural complexity.

Our proposal. Achieve the same coarse-to-fine effect by making the standard dLLM's behaviour **noise-dependent**—no new architecture, no new parameters, no additional inference cost. The key insight is that the noise level t (equivalently, masking ratio $r(t)$) already defines a natural axis from global to local. We make this axis explicit through three orthogonal mechanisms applied at three stages of the forward pass:

Priority	Mechanism	Acts on	What changes
P0	C: Structured Masking	Input	What the model sees
P1	A: Loss Granularity	Output/Loss	What the model is optimised toward
P2	B: Attention Routing	Compute	How the model processes information

Table 1: Three orthogonal mechanisms, ordered by priority.

2 Background: What Block Diffusion Actually Does

Block Diffusion operates in two stages:

1. **Block-level diffusion.** The sequence is partitioned into fixed-size blocks. A diffusion process operates over block representations (e.g., mean-pooled embeddings), generating a coarse "plan" of the entire sequence.

2. **Token-level diffusion.** Conditioned on the block-level plan, a second diffusion process fills in tokens within each block.

The critical structural properties this achieves:

- **Scale decoupling.** Block-level predictions have a receptive field that spans the entire sequence relative to the semantic unit size. Token-level predictions operate locally.

- **Ordered generation.** Global structure is committed before local details.

- **Simplified per-stage task.** Each diffusion process operates at a single granularity, avoiding the need to simultaneously learn global and local patterns.

Our goal is to replicate these three properties within a single-tier dLLM.

3 Mechanism C: Noise-Dependent Structured Masking (P0—Primary)

3.1 The Problem with Uniform Random Masking

In standard dLLM training (e.g., MDLM), at every noise level $r(t)$, each token is independently masked with probability $r(t)$. At high noise ($r = 0.9$) the model sees roughly 10% of tokens scattered randomly. This scattered context does *not* preserve any local coherent structure—the model sees isolated tokens, not coherent phrases or sentences. Consequently, the model's prediction task at high noise is not "plan the global structure given local summaries" but rather "guess tokens from sparse, incoherent fragments." This fails to replicate Block Diffusion's scale decoupling.

3.2 Core Design

Replace uniform random masking with a **noise-dependent structured masking schedule**:

- **High noise** (large $r(t)$): mask entire contiguous spans (sentences, clauses, or fixed-length chunks), but leave some spans fully visible. The model sees a sparse set of *coherent local windows* and must predict the content of missing spans—this is span-level planning.
- **Low noise** (small $r(t)$): gradually transition to standard token-level random masking within spans. The model now does local infilling.

3.3 Formal Specification

Let $r(t) \in [0, 1]$ be the masking ratio at timestep t. Define a **structuredness parameter** $s(t) \in [0, 1]$ that controls the granularity of masking:

$$s(t) = \min\Big(1,\ \frac{r(t) - r_{\text{low}}}{r_{\text{high}} - r_{\text{low}}}\Big)^{+} \tag{1}$$

where $r_{\text{low}}, r_{\text{high}}$ are hyperparameters (e.g., $r_{\text{low}} = 0.3$, $r_{\text{high}} = 0.7$).

The masking procedure at noise level t:

1. Partition the sequence of length L into spans of size $B(t) = \lfloor 1 + s(t) \cdot (B_{\max} - 1) \rfloor$, where $B_{\max}$ is the maximum span size (e.g., sentence length or a fixed value such as 32).
2. For each span, decide to mask the entire span (with probability p_{span}) or leave it fully visible, where p_{span} is calibrated so that the overall masking ratio is $r(t)$.
3. Within masked spans at lower $s(t)$, additionally unmask individual tokens with probability $1 - s(t)$, creating a smooth interpolation toward token-level masking.

At the extremes.

- $s(t) = 1$ (high noise): pure span masking. Model sees coherent blocks, predicts missing blocks. $\approx$ Block Diffusion's block-level stage.
- $s(t) = 0$ (low noise): pure token masking. Standard MDLM behaviour. $\approx$ Block Diffusion's token-level stage.

3.4 Implementation Steps

Step 1: Modify the masking function in the data pipeline. In MDLM's codebase, the masking is typically applied in `noise_schedule.py` or equivalent. The current code performs uniform random masking:

```
# Current: uniform random masking
mask = torch.rand(seq_len) < r_t
```

Replace with:

```
def structured_mask(seq_len, r_t, r_low=0.3, r_high=0.7, B_max=32):
    s_t = max(0, min(1, (r_t - r_low) / (r_high - r_low)))
    span_size = max(1, int(1 + s_t * (B_max - 1)))

    # Partition into spans
    n_spans = seq_len // span_size

    # Decide which spans to mask (calibrate to achieve overall ratio
        r_t)
    span_mask_prob = r_t  # First-order approximation
    span_decisions = torch.rand(n_spans) < span_mask_prob

    # Expand to token level
    token_mask = span_decisions.repeat_interleave(span_size)[:seq_len]

    # Within masked spans, unmask individual tokens with prob (1 - s_t)
    if s_t < 1.0:
        within_span_unmask = torch.rand(seq_len) > s_t
        token_mask = token_mask & ~(token_mask & within_span_unmask)

    return token_mask
```

Step 2: Verify that the effective masking ratio matches $r(t)$. The span-level masking may introduce slight deviations from the target $r(t)$ due to discretisation. Add a calibration step that adjusts `span_mask_prob` to match the target ratio in expectation.

Step 3: Handle span boundaries. For natural language, ideally spans align with sentence or clause boundaries. Two options:

- **Simple (recommended for v1):** Fixed-size spans ($B_{\max} = 32$ tokens). Easy to implement, no dependency on tokenisation.
- **Semantic (optional v2):** Pre-detect sentence boundaries (e.g., by period/newline positions in the token sequence) and use them as span boundaries. Better alignment with semantic units but adds preprocessing complexity.

Step 4: Training. Train the dLLM with the modified masking schedule. Everything else (architecture, optimiser, noise schedule for $r(t)$ over diffusion timesteps) remains identical to the baseline.

Step 5: Inference with structured unmasking. To align inference with the training distribution, use a **noise-dependent structured unmasking schedule** that mirrors the training masking structure. The key idea: reuse the same $s(t)$ function to control unmasking granularity at each denoising step.

- **High noise steps** ($s(t) \approx 1$): instead of unmasking tokens independently by per-token confidence, compute *span-level average confidence* and unmask entire spans whose average confidence exceeds a threshold. This matches the training regime where the model saw and predicted at span granularity.
- **Low noise steps** ($s(t) \approx 0$): revert to standard per-token confidence-based unmasking. This matches the token-level masking the model was trained with at low noise.
- **Intermediate steps:** a soft version—rank by span confidence but allow within-span variation, adding a span-level correlation bias to the unmasking decisions rather than enforcing hard span boundaries.

This structured inference schedule introduces no new parameters and negligible overhead (one average-pooling operation per span per step). It directly addresses the potential train–inference distribution mismatch.

3.5 Key Hyperparameters

Hyperparameter	Role	Suggested range
r_{low}	Below this, masking is purely token-level	0.2–0.4
r_{high}	Above this, masking is purely span-level	0.6–0.8
$B_{\max}$	Maximum span size in tokens	16–64

3.6 Why This Should Work (Connection to Block Diffusion)

Block Diffusion's block-level stage is equivalent to: "given some blocks of text, predict the missing blocks." Mechanism C at high noise does exactly this, but without a separate model or architecture—it is the same model, trained to perform block-level prediction when the noise is high and token-level prediction when the noise is low. The noise level acts as an implicit "which stage am I in?" signal.

4 Mechanism A: Noise-Dependent Loss Granularity (P1—Secondary)

4.1 Motivation

Even with structured masking (C), the training loss at all noise levels is still **token-level cross-entropy**. At high noise, we want the model to learn global structure, but token-level CE may reward local token statistics (e.g., predicting common function words) more than structural coherence. The loss should reflect the granularity of the task.

4.2 Core Design

At high noise levels, augment the standard token-level loss with a **span-level prediction objective** that explicitly rewards global coherence:

$$\mathcal{L}(t) = \mathcal{L}_{\text{token}}(t) + \alpha(t) \cdot \mathcal{L}_{\text{span}}(t) \tag{2}$$

where $\alpha(t) \in [0, 1]$ is a noise-dependent weight:

$$\alpha(t) = s(t) \tag{3}$$

using the same structuredness parameter from Mechanism C.

4.3 What Is the Span-Level Loss?

Several options, ordered by implementation simplicity:

Option A1 (Simplest): Bag-of-words span prediction. For each masked span, pool the model's token-level logits within that span and compute a loss against the bag-of-words distribution of the ground truth span:

$$\mathcal{L}_{\text{span}} = \sum_{\text{masked spans } S} D_{\text{KL}}\big(\text{BoW}(S_{\text{true}}) \parallel \text{MeanPool}(\hat{p}_\theta(S))\big) \tag{4}$$

This is cheap (averaging logits plus a KL term) and explicitly rewards getting the right set of tokens in a span, regardless of order—a planning-level signal.

Option A2 (Moderate): Span embedding prediction. Pass the ground truth span through a frozen sentence encoder (e.g., the model's own last-layer representations pooled) and regress the model's predicted span embedding toward it. This is more semantically meaningful than BoW but requires a pooling and regression head.

Option A3 (Heaviest): Span-level contrastive loss. Treat masked span prediction as a retrieval problem: the model's predicted span embedding should be closer to the ground truth span embedding than to random span embeddings. This is the most expressive but also most expensive.

4.4 Recommended Implementation: Start with A1, Prepare A2

Why A1 is a reasonable starting point despite its crudeness. The role of the span-level loss is not to precisely guide planning, but to provide a better-shaped gradient signal than token-level CE at high noise. When the model sees only 10% of tokens and must predict masked spans, token-level CE rewards guessing individual tokens correctly—which incentivises predicting frequent function words. BoW redirects the optimisation toward "get the right set of content words in this region," which is a strictly better proxy for planning than per-position accuracy, even though it discards ordering information. The two losses also operate in complementary noise regimes ($\alpha(t) \to 0$ at low noise), so gradient conflict is unlikely.

Prepare A2 from the start. If Phase 1 diagnostics show that C-only already improves span-level topic consistency but A1 adds no further gain, the likely explanation is that BoW is too coarse. A2—using the model's own last-layer mean-pooled representations as targets with a lightweight projection head—captures semantic coherence rather than just lexical coverage, and costs only a few dozen lines of additional code. Implement A2 alongside A1 so that switching is a one-line config change rather than a mid-project engineering task.

Step 1. During training, after computing token-level logits $\hat{p}_\theta(x_i \mid x_t)$ for all positions:

```
# Standard token-level CE loss (unchanged)
loss_token = F.cross_entropy(logits, targets, reduction='none')  # (B, L)

# Span-level BoW loss (only at high noise)
if s_t > 0:
    for span_start, span_end in masked_spans:
        # Pool logits within span -> predicted BoW distribution
        span_logits = logits[:, span_start:span_end, :].mean(dim=1)  # (B, V)
        pred_bow = F.softmax(span_logits, dim=-1)

        # Ground truth BoW
        true_tokens = targets[:, span_start:span_end]
        true_bow = token_ids_to_bow(true_tokens, vocab_size)  # (B, V)

        # KL divergence
        loss_span += F.kl_div(
            pred_bow.log(), true_bow, reduction='batchmean')

    loss = loss_token.mean() + alpha_t * loss_span
else:
    loss = loss_token.mean()
```

Step 2: Calibrate $\alpha(t)$. Start with $\alpha(t) = s(t)$ (linear schedule). If the span loss dominates early training, scale it down with a multiplicative constant λ_{span} (e.g., 0.1).

Step 3: Verify token-level quality at low noise. At $s(t) = 0$ the span loss contributes nothing, so low-noise behaviour should be identical to the baseline. Monitor perplexity at low noise levels during training.

4.5 Why A Complements C

C changes what the model *sees* at high noise; A changes what it is *optimised for*. Without A, the model might still exploit structured masking to learn local shortcuts (e.g., predicting easy tokens at span boundaries). A explicitly rewards a holistic span-level prediction, pushing the model toward genuine planning.

5 Mechanism B: Noise-Dependent Attention Routing (P2—Exploratory)

5.1 Motivation

At high noise, most of the input is masked. The model's attention has limited useful context. Uniform full attention may waste capacity attending to uninformative `[MASK]` tokens. By routing attention toward long-range visible tokens at high noise, we can potentially improve the model's ability to capture global dependencies.

5.2 Core Design

Apply a **noise-dependent attention mask** that biases attention toward long-range context at high noise:

- **High noise:** use dilated or strided attention—attend to every k-th position globally, skipping local `[MASK]` tokens.
- **Low noise:** use standard full (or local) attention.

5.3 Specification

For noise level t with structuredness $s(t)$, the attention mask $M(t)$ for query position i and key position j is:

$$M(t)_{ij} = \begin{cases} 1 & \text{if } |i-j| \leq w_{\text{local}} \quad \text{(always attend locally)} \\ 1 & \text{if } x_j \neq [\text{MASK}] \quad \text{(always attend to visible tokens)} \\ s(t) & \text{if } j \bmod k(t) = 0 \quad \text{(dilated global attention)} \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

where $k(t) = \lfloor 1 + s(t) \cdot (k_{\max} - 1) \rfloor$ controls the dilation factor.

5.4 Implementation Concerns

This is P2 (lowest priority) for concrete reasons:

1. **Inference overhead.** Noise-dependent attention masks require dynamic masking at each denoising step. Standard dLLM inference can use static KV caches; this breaks that.
2. **Train–test coupling.** The attention pattern becomes a function of the noise level, adding a new axis of complexity.
3. **Unclear marginal gain.** If C already ensures that visible tokens at high noise are coherent spans, the model can naturally learn to attend to them without forced routing.
4. **Implementation complexity.** Flash Attention and other efficient implementations do not natively support arbitrary dynamic masks.

5.5 When to Pursue B

Include B in the ablation study if:

- C alone shows improvement but the attention analysis reveals that the model still wastes significant attention on `[MASK]` tokens at high noise.
- The target task involves very long sequences (4K+ tokens) where attention efficiency matters.

Otherwise, report B as a negative result in the appendix—this is still valuable.

6 Experimental Plan

6.1 Ablation Structure

Experiment	C	A	B	Purpose
Baseline	×	×	×	Standard dLLM (MDLM/SEDD)
C-only	✓	×	×	Primary mechanism
A-only	×	✓	×	Isolate loss effect
C+A	✓	✓	×	Full method (if additive)
C+A+B	✓	✓	✓	Completeness (likely appendix)

6.2 Evaluation: Tasks Where Planning Matters

The improvement should be most visible on:

1. **Long-form text generation** (1K–4K tokens): stories, articles, reports. Measure global coherence via MAUVE, coherence metrics, and human evaluation.
2. **Multi-step reasoning** (GSM8K, MATH with chain-of-thought): the "plan" is the reasoning structure. Measure accuracy.
3. **Structured generation** (code, outlines, structured documents): planning determines the skeleton. Measure functional correctness or structural metrics.

Tasks where planning *does not* matter (use as controls): short text generation (< 128 tokens), unconditional language modelling perplexity.

6.3 Diagnostic Metrics

Beyond task performance, directly measure whether planning behaviour has improved:

- **Early-step global coherence.** At high noise, sample the partially denoised sequence. Measure topic consistency across segments (e.g., via embedding similarity of revealed spans).
- **Denoising order analysis.** Track which tokens get unmasked first. Does structured masking training shift early-unmasked tokens from function words to content/structural words?
- **Attention distance at high noise.** Measure the average attention distance at high vs. low noise. If C works, the model should show longer attention reach at high noise.

- **Skeleton token survival rate (implicit commitment).** Track the tokens unmasked during the high-noise "planning" steps (the first ∼30% of denoising) and measure what fraction survive to the final output without being overwritten in later steps. A high survival rate indicates that the model's early planning decisions are being respected—i.e., commitment is achieved implicitly through conditioning on already-unmasked tokens. If the survival rate is low, this signals that the single-tier architecture is failing to maintain planning coherence.

- **Span-level semantic consistency at frozen noise levels.** At several noise checkpoints during inference, freeze the model's output and evaluate span-level predictions for semantic coherence (e.g., do the predicted content words for adjacent masked spans form a topically consistent narrative?). This directly tests whether Mechanism C is inducing genuine planning rather than independent per-span prediction.

7 Implementation Roadmap

Phase 1: Mechanism C (Weeks 1–3)

Week 1

Implement structured masking in MDLM codebase. Implement structured inference schedule (span-level confidence unmasking). Verify masking ratio calibration. Train small-scale model (128M params) on OpenWebText.

Week 2

Evaluate on text generation benchmarks. Run denoising order analysis and skeleton token survival rate diagnostic. Compare against uniform masking baseline.

Week 3

Hyperparameter sweep over r_{low}, r_{high}, $B_{\max}$. Run span-level semantic consistency diagnostic at frozen noise levels.

Phase 2: Mechanism A (Weeks 4–5)

Week 4

Implement both A1 (BoW) and A2 (span embedding) losses. Train C+A1 variant. Compare against C-only.

Week 5

If A1 gain is significant, tune $\alpha(t)$ schedule. If not, switch to A2 (should be a config change) and evaluate. Use span-level topic consistency to decide.

Phase 3: Mechanism B + Paper (Weeks 6–8)

Week 6

Run B ablation (one setting only, no extensive tuning). Conduct attention analysis.

Weeks 7–8

Scale to larger model if results are positive. Write paper.

8 Risk Assessment

Risk	Severity	Mitigation
Structured masking introduces train–test mismatch in denoising order	High	Use a structured inference schedule that mirrors training: at high-noise steps, unmask by span-level average confidence; at low-noise steps, revert to per-token confidence. Reuse the same $s(t)$ function. See Section 3.4, Step 5.
BoW span loss is too crude to capture planning	Medium	A1 redirects gradients from per-token guessing toward region-level content prediction—a strictly better signal than token CE at high noise even if coarse. Implement A2 in parallel from the start so switching is a config change.
Lack of explicit commitment mechanism	Medium	At low noise the model is conditioned on tokens already unmasked in high-noise steps, so planning information is carried implicitly. Validate via skeleton token survival rate (Section 6.3). Only add explicit constraints if survival rate is low.
Gains are marginal over well-tuned baseline	High	Choose evaluation tasks where planning matters (long-form, multi-step). Small gains on short text are expected.
Structured masking slows training throughput	Low	Span masking is a simple index operation, negligible overhead vs. the transformer forward pass.
Block Diffusion's gain was from something other than scale decoupling	Medium	Run a diagnostic: compare our C against a "block-conditioned" baseline that uses Block Diffusion's plan but our single-tier model.